Семинар 3. Pthreads: барьеры и условные переменные

Барьер — это фрагмент кода, который характеризуется тем, что ни один из потоков не выйдет из него, пока последний не зашел.

1 Холостой цикл

Самый простой способ организации барьера — это использование холостого цикла. Минусы такого подхода:

1. Значительное падение производительности в случае, когда количество запущенных потоков превышает количество физических. Это связано с тем, что проходит значительное время, прежде чем операционная система догадается, что поток ничего не делает и висит в бездействии, и отключит его, чтобы дать возможность поработать другим потокам.

2. Компилятор не знает, что мы пишем многопоточную программу. Если включить оптимизацию (использовать флаг `-O2`), то компилятор может переставить некоторые строки, которые, с его точки зрения, не зависят друг от друга, и тогда программа перестанет работать.

3. Мы не можем синхронно обнулить `counter`, поэтому эта переменная является одноразовой. Нам придется создавать массив счетчиков, если барьеров несколько.

Листинг 1: Организация барьера с помощью холостого цикла

```
#include <pthread.h>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include "timer.h"

#define NTHREADS 4
#define NBARRIERS 10000
pthread_mutex_t mutex;
int counter[NBARRIERS];

void *routine(void *rank){
  for (int j = 0; j < NBARRIERS; j++){
      pthread_mutex_lock(&mutex);
    counter[j]++;
    pthread_mutex_unlock(&mutex);
    while(counter[j] != NTHREADS);
  }
  return NULL;
}

int main(){
  pthread_t thread_handles[NTHREADS];
  pthread_mutex_init(&mutex, NULL);
  double start, end;
  GET_TIME(start);
  for (int i = 0; i < NTHREADS; i++)
    pthread_create(&thread_handles[i], NULL, routine, NULL);

  for (int i = 0; i < NTHREADS; i++)
    pthread_join(thread_handles[i], NULL);
  GET_TIME(end);
  printf("%e\n", end - start);
  pthread_mutex_destroy(&mutex);
}
```

2 Семафоры

Листинг 2: Организация барьера с помощью семафоров

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "timer.h"
5  #include <semaphore.h>
6
7  #define NTHREADS 4
8  #define NBARRIERS 10000
9  pthread_mutex_t mutex;
10 sem_t sem_barrier[NBARRIERS];
11 sem_t sem_count;
12 int counter = 0;
13
14 void *routine(void *rank){
15   for (int j = 0; j < NBARRIERS; j++){
16     sem_wait(&sem_count);
17     if (counter == NTHREADS - 1){
18       counter = 0;
19       sem_post(&sem_count);
20       for (int i = 0; i < NTHREADS - 1; i++)
21         sem_post(&sem_barrier[j]);
22     }
23     else{
24       counter++;
25       sem_post(&sem_count);
26       sem_wait(&sem_barrier[j]);
27     }
28   }
29   return NULL;
30 }
31
32 int main(){
33   pthread_t thread_handles[NTHREADS];
34   pthread_mutex_init(&mutex, NULL);
35   for(int i=0; i<NBARRIERS; i++)
36     sem_init(&sem_barrier[i], 0, 0);
```

```
37    sem_init(&sem_count, 0, 1);
38    double start, end;
39    GET_TIME(start);
40    for (int i = 0; i < NTHREADS; i++)
41      pthread_create(&thread_handles[i], NULL, routine, NULL);
42
43    for (int i = 0; i < NTHREADS; i++)
44      pthread_join(thread_handles[i], NULL);
45    GET_TIME(end);
46    printf("%e\n", end - start);
47    for(int i=0; i<NBARRIERS; i++)
48      sem_destroy(&sem_barrier[i]);
49    sem_destroy(&sem_count);
50    pthread_mutex_destroy(&mutex);
51  }
```

По прохождению барьера в программе 2 обнуляется `counter`, а семафор `sem_count` опять принимает значение 1. Однако семафор `barrier_sem` может не занулиться. Именно, может реализоваться следующий сценарий: потоки, все, кроме последнего, зашедшего в барьер, уснут в строке 31. Операционная система, заметив бездействие потоков, временно прекратит их работу. Последний поток в 25 строке поднимет семафор, пробуждая зависшие потоки. Однако, если потоки отключены от планировщика задач, то они не увидят поднятых семафоров. Тогда последний поток, подняв семафор некоторое количество раз, продолжит работу, а значение `barrier_sem` будет отлично от нуля. В этом случае последний поток продолжит работу, дойдет до второго барьера, а т.к. `barrier_sem` не занулен, то этот поток пойдет дальше второго барьера, к третьему, а зависшие в первом барьере потоки так его и не преодолеют.

Следовательно, барьерный семафор является одноразовым. Его нельзя переиспользовать.

3 Условные переменные

Для реализации барьера, которая была бы лишена выявленных выше недостатков, будем использовать новый тип данных — условную переменную, `pthread_cond_t`.

Инициализация условной переменной

```
int pthread_cond_init(
  pthread_cond_t*    cond_p,     /*out*/
  pthread_condattr_t  cond_attr    /*in*/
);
```

Освобождение ресурсов, занимаемых условной переменной

```
int pthread_cond_destroy(
  pthread_cond_t*    cond_p      /*in/out*/
);
```

Условная переменная используется для того, чтобы сигнализировать потокам о том, что событие, которое символизирует данная условная переменная, произошло.

Условная переменная имеет тип `pthread_cond_t`

Условная переменная используется всегда в связке с мьютексом. Пример использования условной переменной для реализации барьера показан в листинге 3.

Обратим внимание на то, что в строке 29 вызов функции `pthread_cond_wait` поставлен в цикл `while`. Это сделано для того, чтобы отличить «пробуждение» потока вследствие срабатывания условной переменной от самопроизвольного пробуждения, вызванного другими причинами (spurious wakeup).

Процессы ждут связанного с условной переменной события в функции `pthread_cond_wait`:

```
pthread_cond_wait(
        pthread_cond_wait* cond,          /*in/out*/
        pthread_mutex_t* mutex            /*in/out*/
);
```

Функция `pthread_cond_wait` производит следующие действия:

```
pthread_mutex_unlock(&mutex);
wait_on_signal(&cond);
pthread_mutex_lock(&mutex);
```

Для того, чтобы сигнализировать одному какому-то потоку, что событие, связанное с условной переменной, произошло, используется функция

```
pthread_cond_signal(
        pthread_cond_t* cond      /*in/out*/
```

```
);
```

При вызове этой функции какой-то один из потоков, заблокированных в `pthread_cond_wait` (точнее, в ее части `wait_on_signal`), проснется, и продолжит выполнение своих инструкций. Если же мы хотим пробудить все уснувшие потоки, то нужно использоваться функцию

```
pthread_cond_broadcast(
        pthread_cond_t* cond      /*in/out*/
);
```

Таким образом, общую схему работы с условными переменными можно описать следующим псевдокодом:

```
lock mutex
if condition has occured
        signal thread(s)
else
        unlock mutex
        wait_on_signal
        lock mutex
unlock mutex
```

Листинг 3: Барьер с использованием условных переменных

```
 1 #include <pthread.h>
 2 #include <stdio.h>
 3 #include <stdlib.h>
 4 #include "timer.h"
 5 #include <semaphore.h>
 6
 7 #define NTHREADS 4
 8 #define NBARRIERS 10000
 9 pthread_mutex_t mutex;
10 int counter;
11 pthread_cond_t cond;
12
13 void *routine(void *rank){
14   for (int j = 0; j < NBARRIERS; j++){
15     pthread_mutex_lock(&mutex);
16     counter++;
```

```
17      if (counter == NTHREADS){
18        counter = 0;
19        pthread_cond_broadcast(&cond);
20      }
21      else while(pthread_cond_wait(&cond, &mutex) != 0);
22      pthread_mutex_unlock(&mutex);
23    }
24    return NULL;
25  }
26
27  int main(){
28    pthread_t thread_handles[NTHREADS];
29    pthread_cond_init(&cond, NULL);
30    pthread_mutex_init(&mutex, NULL);
31    double start, end;
32    GET_TIME(start);
33    for (int i = 0; i < NTHREADS; i++)
34      pthread_create(&thread_handles[i], NULL, routine, NULL);
35    for (int i = 0; i < NTHREADS; i++)
36      pthread_join(thread_handles[i], NULL);
37    GET_TIME(end);
38    printf("%e\n", end - start);
39    pthread_cond_destroy(&cond);
40    pthread_mutex_destroy(&mutex);
41  }
```

В качестве еще одного примера рассмотрим применение условных переменных к задаче передачи сообщений по кругу, см. листинг 4.

Листинг 4: Использование условных переменных для циклической передачи сообщений

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

const int MAX_THREADS = 1024;
const int MSG_MAX = 100;

int thread_count;
```

```
char** messages;
pthread_cond_t* cond;
pthread_mutex_t mutex;

void *Send_msg(void* rank);

int main(int argc, char* argv[]) {
        long        thread;
        pthread_t* thread_handles;

        thread_count = strtol(argv[1], NULL, 10);

        thread_handles = malloc (thread_count*sizeof(pthread_t));
        messages = malloc(thread_count*sizeof(char*));
        cond = malloc(thread_count*sizeof(pthread_cond_t));
        pthread_mutex_init(&mutex, NULL);
        for (thread = 0; thread < thread_count; thread++) {
                messages[thread] = NULL;
                pthread_cond_init(&cond[thread], NULL);
        }

        for (thread = 0; thread < thread_count; thread++)
                pthread_create(&thread_handles[thread], NULL,
                        Send_msg, (void*) thread);

        for (thread = 0; thread < thread_count; thread++) {
                pthread_join(thread_handles[thread], NULL);
        }

        for (thread = 0; thread < thread_count; thread++) {
                free(messages[thread]);
                pthread_cond_destroy(&cond[thread]);
        }
        pthread_mutex_destroy(&mutex);
        free(messages);
        free(cond);
        free(thread_handles);
```

```
}

void *Send_msg(void* rank) {
        long my_rank = (long) rank;
        long dest = (my_rank + 1) % thread_count;
        char* my_msg = (char*) malloc(MSG_MAX*sizeof(char));

        sprintf(my_msg, "Hello to %ld from %ld", dest, my_rank);
        messages[dest] = my_msg;

        pthread_mutex_lock(&mutex);
        pthread_cond_signal(&cond[dest]);
        if (messages[my_rank] == NULL)
                pthread_cond_wait(&cond[my_rank],&mutex);
        pthread_mutex_unlock(&mutex);


        printf("Thread %ld > %s\n", my_rank, messages[my_rank]);

        return NULL;
}
```